

# AMATH 482/582: HOMEWORK 5

JONATHAN MCCORMACK

*Applied Mathematics Department, University of Washington, Seattle, WA*  
*jrmack20@uw.edu*

ABSTRACT. Politicians in the United States are predominately members of the Democrat and Republican parties. Because all congressmen identify with one of these two parties, we hope to generalize the voting patterns of these two groups, by conducting unsupervised learning on the votes made by the members of the 1984 congress on 16 different votes. Using Spectral clustering, we generated an unnormalized graph Laplacian matrix, whose Fiedler vector showed the consensus of each party on any particular issue. We optimized the weight of the graph Laplacian matrix using a Gaussian Distribution with  $\sigma = 1.16$  providing an accuracy of 0.88. We then used the optimized graph Laplacian matrix to conduct semi-supervised learning and generate a affine function that predicts the political party of a random individual using their votes on the same 16 issues. This affine function had an accuracy of 0.89 when built using  $\sigma = 1.16$ .

## 1. INTRODUCTION AND OVERVIEW

As machine learning becomes increasingly ubiquitous, certain human industries, such as politics, lack the 'neat' labeling useful for simple, supervised learning regression algorithms. Unsupervised learning allows insights to be derived by allowing unlabeled connections between data to be revealed by treating each sample (politician) as a node in a graph, where closeness and similarity between samples may have deeper levels of meaning.

In this study, we analyzed the 1984 House voting records data, where the 435 members of the House vote on 16 different bills. The House is comprised of 267 democrats and 168 republicans. This paper intends to explore how distinct the two political parties are from each other using unsupervised learning. Then use semi-supervised learning to created a different model, to see if knowing the labeling of the data improves the model's accuracy.

## 2. THEORETICAL BACKGROUND

This study works with unsupervised and semi-supervised learning. Unsupervised learning uses feature data without known classifications of the individual samples, where semi-supervised and supervised have at least some knowledge of the output classifications of interest.

**2.1. Spectral Clustering.** Clustering is a general concept in unsupervised learning of discovering and defining different centers of gravity in a set of data. It is especially useful in visualizing the 'spread' of the different samples, where there may be meaningful relationships that supervised learning could ignore. Spectral clustering is a robust, clustering method that applies graph theory to cluster data based on underlying relationships rather than simple euclidean distance[1].

In order to identify the underlying relationships, we need to establish the similarity or nearness between any two samples. We measure the similarity by using the weight (adjacency) matrix, a

matrix that evaluates the weighted, 2-norm of the pairwise distance between every combination of sample vectors. Given a data set  $\mathbf{X} = \{x_0, \dots, x_{N-1}\} \in \mathbb{R}^d$ , the entries in the weight matrix are given by  $w_{ij} = \eta(\|\mathbf{x}_i - \mathbf{x}_j\|_2)$ . Each row now represents the  $i$ th sample vector, whose column entries are the respective weights of that row's vector with the  $j$ th sample vector. The  $\eta$  function is given by the below function (1). The  $\eta$  function is governed by the parameter  $\sigma$ , which affects the sharpness of the distribution of the weight. A small  $\sigma$  value will consider most data to be unrelated, whereas a large  $\sigma$  value will tend to exaggerate the closeness between otherwise unrelated samples. In the computational results section below, we demonstrate the tuning of this parameter to increase accuracy.

$$(1) \quad \eta = \exp\left(-\frac{t^2}{2\sigma^2}\right)$$

Combined with the above  $\eta$  function, the weight matrix becomes the below function (2).

$$(2) \quad w_{ij} = \exp\left(-\frac{\|\mathbf{x}_i - \mathbf{x}_j\|_2^2}{2\sigma^2}\right)$$

It is important to note, that if two sample vectors are identical, the weight is 1, and the weight decreases as sample vectors become more dissimilar. Next, we want to create the degree vector,  $d \in \mathbb{R}^N$ , whose entries are the column sums of the weight matrix, that is  $d_j = \sum_{i=0}^{N-1} W_{ij}$ . The degree vector is then expanded to form the degree matrix(3). Here it is important to note that the sample vectors that are the most central will have the highest degree because they have the highest column sum.

$$(3) \quad = \text{diag}(d) = \begin{bmatrix} d_0 & & & \\ & d_1 & & \\ & & \ddots & \\ & & & d_{N-1} \end{bmatrix}$$

With the degree matrix and the weight matrix, we form the unnormalized Graph Laplacian matrix(4).

$$(4) \quad \tilde{L} = D - W$$

The unnormalized Graph Laplacian matrix has a number of useful properties. First, it is symmetric because both the weight and degree matrices are symmetric. It is positive, semi-definite, meaning all eigenvalues are greater than or equal to zero. Because, all row sums and column sums are 0, a feature of how the degree matrix was derived from the weight matrix, the Graph Laplacian Matrix must have a zero eigenvalue, whose eigenvector is the ones vector. When one usually performs an eigendecomposition, the largest eigenvalue's eigenvector represents the largest most representative component of all entries in the matrix, this represents the centrality of the whole dataset. This eigenvector 'cuts' through the largest number of sample vectors. However, the second smallest eigenvalue's eigenvector, the Fiedler vector, is a vector that 'cuts' through the least number of sample vectors, making the best cut to divide the sample into two distinct clusters. We then take the sign of the Fiedler vector (positive or negative) and treat that as the 'classification' of that row of the original data set. Since the output vector from the training data, if provided, should have been coded into two separate classes (positive or negative), this Fiedler vector can be compared to known output data for an accuracy assessment. Simple accuracy was calculated using  $\text{Accuracy} = 1 - \left(\frac{\#\text{misclassified}}{\#\text{points}}\right)$ .

**2.2. Semi-Supervised Learning.** Semis-supervised learning is a technique that combines the unsupervised spectral clustering, with the ability to leverage known outputs in supervised learning. Given the same input data as before, with the additional output vector, we build the same Graph Laplacian Matrix. However, we use the eigenvectors as a feature matrix or 'Laplacian' embedding of the original data matrix. This matrix,  $A$ , is the first  $M$  eigenvectors of  $\tilde{L}$  using only the first  $J$  rows. We then create a modified output vector,  $b$ , with only the first  $J$  output values. Then, we use  $A$  and  $b$  to solve for the coefficient vector,  $x$ , using the linear regression equation below(5). The coefficients should find a relationship between the rows of the eigenvalues and the outputs of each sample (row). Therefore, we should be able to create a prediction,  $\hat{y}$ , by solving  $\hat{y} = Ax$ , where  $A$  is now the same  $M$  eigenvalues, but all rows of the  $\tilde{L}$  matrix. The sign of the  $\hat{y}$  vector should be compared to the true output vector in the same manner as the Fiedler vector.

$$(5) \quad \hat{x} = \operatorname{argmin}_x \|Ax - b\|_2^2$$

### 3. ALGORITHM IMPLEMENTATION AND DEVELOPMENT

The code took advantage of the numpy, scipy, matplotlib, and scikit-learn packages, specifically the Ridge function. The data was preprocessed separating the feature (votes) array from the output vector (political party). The votes were labeled +1, -1, or 0 for yes, no, or other. The political parties were labeled +1 or -1 for republican or democrat. We sorted the data with democrats then republicans for the unsupervised learning, specifically for plotting, but left the data unsorted for semi-supervised learning.

### 4. COMPUTATIONAL RESULTS

In Spectral clustering, we defined a general function for calculating the adjacency (weight) matrix given some  $\sigma$  value and produces the unnormalized graph Laplacian matrix. We iterated calculated the eigendecompositions of the matrices generated for  $\sigma$  values rangin from 0 to 4. Then we compared the accuracy of the Fiedler vectors to the original output data. Because the Fiedler vector shows the strength of the association, we only considered the sign of the vector; that is we treated any positive value as +1 (republican) and any negative value as -1 (democrat). Because the Fiedler vector shows association, but does not indicate positive or negative correlation, if the accuracy was less than 0.5, we assumed the algorithm had the positives and negatives reverse. Therefore we changed the accuracy to 1 - the original accuracy. The optimal  $\sigma$  was found to 1.16 achieving an accuracy of 0.88. From figure 1 we can see that as  $\sigma$  increases, the accuracy increases till it hits a peak and stabilizes.

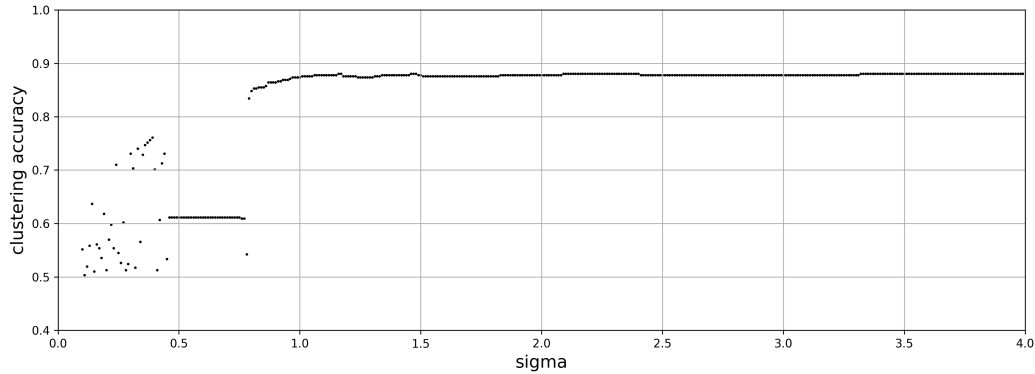


FIGURE 1. Unsupervised learning clustering accuracy versus sigma values

To show how effective the spectral clustering was at predicting the output vector, figure 2 plots the original output vector with -1 (blue democrats) and +1 (red republicans) alongside the predictions made by the Fiedler vector using  $\sigma = 1.16$ . We did not plot the strength of the association, just the sign of the vector, but that may be useful for future analysis and model refinement. The model error could be caused by an over simplified model that does not account for the various subsets of political parties. However, a better model may also remove the votes that obfuscate the underlying similarities between politicians from the same party.

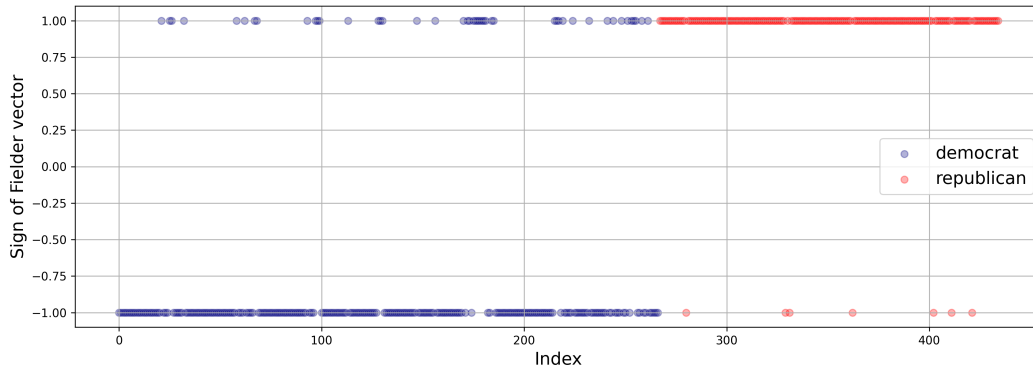


FIGURE 2. The true and predicted parties of each politician. The data is sorted such that the first 267 samples should be -1 (democrat) and the remaining samples should be +1 (republican). The blue +1s and the red -1s are the few politicians incorrectly modeled by unsupervised learning.

In the semi-supervised case, we only produced the optimized graph Laplacian matrix, but then iterated on the number of eigenvectors ( $M$ ) and the number of data rows ( $J$ ) for producing the various affine regressions and their respective accuracies. The regression was performed using a truncated output vector with the same number of outputs and the number of inputs ( $J$ ). To calculate the accuracy, the inner product of the first  $M$  eigenvectors (sorted from smallest to largest eigenvalue) with the regression's coefficient vector. The sign of the output vector is the predicted output vector of the whole data set. In the below table, the accuracy varies very little between 0.7287 and 0.8920. The best accuracy occurred with  $M=6$  and  $J=5$ , but this could have

been improved with an increased value of  $\sigma$ . The value of  $\sigma$  from unsupervised learning is tuned to the lowest error with the unsupervised model, but not to the value of  $\sigma$  that produces the best graph Laplacian matrix.

| J \ M | 2     | 3     | 4     | 5     | 6     |
|-------|-------|-------|-------|-------|-------|
| 5     | 88.97 | 88.97 | 83.91 | 86.67 | 89.20 |
| 10    | 88.74 | 82.07 | 85.75 | 75.40 | 72.87 |
| 20    | 88.28 | 82.30 | 86.44 | 84.14 | 87.59 |
| 40    | 88.05 | 83.91 | 87.59 | 87.82 | 86.44 |

TABLE 1. Summary of accuracy of the predicted output of the affine classifier created using the eigenvectors from the graph Laplacian matrix with  $\sigma = 1.16$ .  $M = 2, 3, 4, 5$ , and  $6$  (the number of eigenvectors) and  $J = 5, 10, 20$ , and  $40$  (the number of rows of data in the eigenvectors).

## 5. SUMMARY AND CONCLUSIONS

This study tested the accuracy of spectral clustering and semi-supervised learning to predict the political party of congressmen of the 1984 House based on their voting records. We created the graph Laplacian matrix, computed eigendecompositions, and classified the individuals based on the Fiedler vector. Spectral clustering found the optimal value of  $\sigma = 1.16$  with an accuracy of 0.88. Semi-supervised learning found the highest accuracy to be 0.89 when  $M = 6$  and  $J = 5$ . Both methods were highly accurate.

In future studies, the most interesting extension would be to increase the number of clusters (political parties) while also removing random votes. This could identify key issues where some politicians may be in a smaller subset of their political party that typically votes with the party, but diverges on some niche topic. This more accurately encompasses how politicians have a breadth and depth of constituents 'weighting' their votes.

## ACKNOWLEDGEMENTS

I would like to thank Professor Hosseini for his lectures, notes, and code examples.

## REFERENCES

- [1] S. L. Brunton and J. N. Kutz. *Data-Driven Science and Engineering: Machine Learning, Dynamical Systems, and Control*. Cambridge University Press, 2 edition, 2022.